COS202

PROJECT REPORT

Mathematical Calculator (MC) & Personal Pocket CGPA Calculator (PPC)

Submitted in fulfilment of the COS202
Friday-to-Monday Coding Assignment (01/07/2026)

Student Name: AKINOLA MICHAEL OBALOLUWA

Matriculation Number: 241205001

Department: Information Systems

Date of Submission: 4th of July, 2026

Source code hosted on GitHub — see Appendix A for repository details.

Table of Contents

1. Objective

The objective of this project is to design, develop, and document two interactive Python console applications in fulfilment of the COS202 Friday-to-Monday coding assignment: a Mathematical Calculator (MC) capable of performing basic arithmetic operations, and a Personal Pocket CGPA Calculator (PPC) that computes a student's GPA, CGPA, and class of degree using selection control statements. A further objective is to host the source code and supporting documentation in a public GitHub repository, demonstrating competence in version control as part of a semester-long portfolio of practical work.

2. Introduction

Calculators and academic performance tools are two of the most common categories of software encountered by beginner programmers, and they serve as excellent vehicles for practising fundamental programming constructs such as loops, conditionals, functions, and error handling. This project responds to a two-part assignment: first, to build a simple, screen-based mathematical calculator; and second, to build a personal, portable CGPA calculator that removes the need for a student to repeatedly consult an external academic records system (such as AVERS) in order to know their standing.

Both applications are written in Python 3, chosen for its readability, its strong support for interactive console programs, and its suitability for demonstrating selection control statements (if / elif / else) as explicitly required for the CGPA calculator. The two programs are kept intentionally simple in scope — the calculator is not a scientific calculator, and the CGPA tool assumes a standard 5-point Nigerian university grading scale — while still satisfying every stated requirement of the assignment.

3. System Requirements

The following are the minimum requirements needed to run both applications:

- Python 3.8 or later (no external/third-party packages are required).
- A terminal, command prompt, or any IDE capable of running Python scripts (e.g. VS Code, PyCharm, IDLE).
- A GitHub account, for hosting the source code repository.
- Git installed locally (optional, but recommended for command-line uploads).

Because both programs rely solely on Python's built-in standard library (input(), print(), and basic arithmetic operators), there are no installation dependencies beyond the Python interpreter itself, which keeps the solution lightweight and portable across Windows, macOS, and Linux.

4. Program Design

4.1 Mathematical Calculator (MC) — Design Overview

The calculator is designed around a menu-driven loop. On start-up, the program displays a header and a menu of available operations. The user selects an operation using its associated symbol, supplies two

numbers, and receives the computed result. The loop continues indefinitely until the user enters 'OFF', at which point the program terminates gracefully.

Each arithmetic operation is implemented as an independent function (add, subtract, multiply, divide, floor_divide, exponent, modulus), and these functions are registered in a dictionary that maps each operator symbol to its corresponding function and display name. This design avoids a long chain of if/elif statements, keeps the code modular, and makes it simple to extend the calculator with new operations in the future.

4.2 Personal Pocket CGPA Calculator (PPC) — Design Overview

The PPC program first asks the user how many courses they wish to enter. It then loops that many times, collecting a course title, credit unit, and score for each course. Each score is converted into a letter grade and grade point using a chain of if/elif/else statements, matching the standard 5-point grading scale. The quality point for each course (grade point multiplied by credit unit) is accumulated, along with the total credit units, to compute the overall GPA. The classification of degree (First Class, Second Class Upper, etc.) is likewise determined using if/elif/else statements applied to the computed CGPA.

5. Algorithms

5.1 Mathematical Calculator Algorithm

Step 1: Display the calculator header and menu of operations.

Step 2: Prompt the user to select an operation, or C, or OFF.

Step 3: If the user selects C, clear the screen and return to Step 1.

Step 4: If the user selects OFF, display a goodbye message and terminate the program.

Step 5: If the user selects a valid operator, prompt for two numeric inputs, validating each until a proper number is entered.

Step 6: Perform the selected calculation. If the operation is division, floor division, or modulus and the second number is zero, display an error message instead of crashing.

the second number is zero, display an error message instead of crashing.

Step 7: Display the result and return to Step 1.

5.2 Personal Pocket CGPA Calculator Algorithm

Step 1: Prompt the user for the number of courses (must be a positive whole number).

Step 2: For each course, prompt for the course title, credit unit, and score (0–100), validating each input.

Step 3: Convert each score to a letter grade and grade point using if/elif/else selection statements.

Step 4: Compute the quality point for the course as: Quality Point = Grade Point × Credit Unit.

Step 5: Accumulate the total quality points and total credit units across all courses.

Step 6: Compute GPA = Total Quality Points / Total Credit Units, and treat CGPA as equal to GPA in this single-semester version.

Step 7: Determine the class of degree from the CGPA using if/elif/else selection statements.

Step 8: Display a full tabulated summary of all courses, followed by the GPA, CGPA, and class of degree.

6. Source Code Explanation

6.1 Mathematical Calculator (mc.py)

The source code below is organised into three clear sections: helper/display functions (for printing the header, menu, and validating numeric input), the core arithmetic functions (one function per operation), and the main program loop that ties everything together. Every function and non-trivial line carries an inline comment explaining its purpose, so the listing below is self-documenting for a beginner reader.

```
"""
=============================================================================
COS202 ASSIGNMENT - SOLUTION 2
PROJECT     : Mathematical Calculator (MC)
LANGUAGE    : Python 3
DESCRIPTION : A simple, interactive, screen-based calculator that
              supports Addition, Subtraction, Multiplication,
              Division, Floor Division, Exponentiation, Modulus,
              Clear, and Off (Exit) operations.
=============================================================================
"""

#
```

```
# ------------------------------------------------------------------
# SECTION 1: HELPER / DISPLAY FUNCTIONS
# ------------------------------------------------------------------

def show_header():
    """
    Prints the calculator's title banner.
    This runs once when the program starts, and again whenever
    the screen is 'cleared' (C option), to keep the interface
    looking clean and professional.
    """
    print("=" * 50)
    print(" SIMPLE MATHEMATICAL CALCULATOR (MC)".center(50))
    print("=" * 50)

def show_menu():
    """
    Displays the list of operations available to the user.
    Keeping this in its own function means we can call it
    repeatedly inside the main loop without repeating code
    (this follows the DRY principle - Don't Repeat Yourself).
    """
    print("\nAvailable Operations:")
    print(" +   -> Addition")
    print(" -   -> Subtraction")
    print(" *   -> Multiplication")
    print(" /   -> Division")
    print(" \\   -> Floor Division (whole-number division)")
    print(" ^   -> Exponent (power)")
    print(" %   -> Modulus (remainder)")
    print(" C   -> Clear screen")
    print(" OFF -> Exit the calculator")
    print("-" * 50)

def get_number(prompt):
    """
    Safely collects a numeric value from the user.

    WHY THIS FUNCTION EXISTS:
    Users can accidentally type letters or symbols instead of
    numbers. Rather than letting the program crash, we use a
    loop + try/except block to keep asking until valid input
    is given. This is what the assignment calls 'proper input
```



```
    validation'.
    """
    while True:
        raw_value = input(prompt).strip()
        try:
            # Try converting to a float first so decimals (e.g. 3.5)
            # are supported, not just whole numbers.
            return float(raw_value)
        except ValueError:
            print(" [Error] That is not a valid number. Please try again.")

# ------------------------------------------------------------------
# SECTION 2: CORE ARITHMETIC OPERATIONS
# ------------------------------------------------------------------
```

```
# Each operation lives in its own small function. This is called
# "modular" programming: every function does ONE clear job, which
# makes the code easier to read, test, and maintain.

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    """
    Regular division. We must guard against division by zero,
    which is mathematically undefined and would otherwise crash
    the program with a ZeroDivisionError.
    """
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero.")
    return a / b

def floor_divide(a, b):
    """
    Floor division (the assignment's '\\' symbol) returns the
    whole-number part of a division, discarding the remainder.
    Example: 7 // 2 = 3
    """
    if b == 0:
        raise ZeroDivisionError("Cannot floor-divide by zero.")
    return a // b

def exponent(a, b):
    """Raises 'a' to the power of 'b' (a ** b)."""
    return a ** b

def modulus(a, b):
    """
    Returns the remainder after dividing 'a' by 'b'.
    Example: 7 % 2 = 1
    """
    if b == 0:
        raise ZeroDivisionError("Cannot perform modulus with zero.")
    return a % b

# ---------------------------------------------------------------------
# SECTION 3: MAIN PROGRAM LOOP
# ---------------------------------------------------------------------

def main():
```



```
    """
    This is the entry point of the program. It repeatedly shows
    the menu, reads the user's chosen operation, performs the
    calculation, and loops back - until the user selects OFF.
    """
    # Dictionary mapping each symbol to its function.
    # This avoids a long chain of if/elif statements and makes
```

```
    # This avoids a long chain of if/elif statements and makes
    # it easy to add new operations later.
    operations = {
        "+": ("Addition", add),
        "-": ("Subtraction", subtract),
        "*": ("Multiplication", multiply),
        "/": ("Division", divide),
        "\\": ("Floor Division", floor_divide),
        "^": ("Exponent", exponent),
        "%": ("Modulus", modulus),
    }

    show_header()

    while True:
        show_menu()
        choice = input("Select an operation (or C / OFF): ").strip()

        # ---- Handle CLEAR ----
        if choice.upper() == "C":
            # Clear the terminal screen for a fresh look.
            print("\n" * 50)  # portable "clear" that works everywhere
            show_header()
            continue

        # ---- Handle OFF (Exit) ----
        if choice.upper() == "OFF":
            print("\nThank you for using the Mathematical Calculator. Goodbye!")
            break

        # ---- Handle a valid arithmetic operation ----
        if choice in operations:
            name, func = operations[choice]
            num1 = get_number("Enter the first number: ")
            num2 = get_number("Enter the second number: ")
            try:
                result = func(num1, num2)
                print(f"\n [{name}] Result: {num1} {choice} {num2} = {result}")
            except ZeroDivisionError as error:
                # Friendly, specific error handling as required by
                # the assignment (division by zero).
                print(f"\n [Error] {error}")
        else:
            # ---- Handle invalid menu input ----
            print("\n [Error] Invalid choice. Please select a valid option "
                  "from the menu.")

# Standard Python entry-point guard: this ensures main() only runs
# when this file is executed directly (not when imported elsewhere).
if __name__ == "__main__":
    main()
```

Listing 1 — mc.py (full source)

6.2 Personal Pocket CGPA Calculator (ppc.py)

The PPC source code follows the same structure: input-validation helpers, the grading logic (implemented purely with if/elif/else selection statements as required by the assignment), and a main function that orchestrates data collection, calculation, and the final tabulated summary.

```
"""
================================================================================
COS202 ASSIGNMENT - SOLUTION 3
PROJECT      : Personal Pocket CGPA Calculator (PPC)
LANGUAGE     : Python 3
DESCRIPTION  : Uses SELECTION CONTROL STATEMENTS (if / elif / else)
               to convert course scores into grade points, then
               calculates Quality Points, Total Credit Units,
               GPA, CGPA, and the resulting Class of Degree -
               all from the palm of your hand, without visiting
               AVERS.

GRADING SCALE: Standard 5-point Nigerian university scale
               A (70-100) = 5.0
               B (60-69)  = 4.0
               C (50-59)  = 3.0
               D (45-49)  = 2.0
               E (40-44)  = 1.0
               F (0-39)   = 0.0
================================================================================
"""


# ------------------------------------------------------------------
# SECTION 1: INPUT VALIDATION HELPERS
# ------------------------------------------------------------------

def get_positive_int(prompt):
    """
    Repeatedly asks the user for a whole number greater than zero.
    Used for 'number of courses' and 'credit unit', where negative
    or non-numeric values would not make sense.
    """
    while True:
        raw_value = input(prompt).strip()
        try:
            value = int(raw_value)
            if value <= 0:
                print(" [Error] Please enter a whole number greater than zero.")
                continue
            return value
        except ValueError:
            print(" [Error] That is not a valid whole number. Try again.")

def get_score(prompt):
    """
    Repeatedly asks for a course score and ensures it is a number
    between 0 and 100 (inclusive), since scores outside that range
    are not valid academic scores.
    """
    while True:
        raw_value = input(prompt).strip()
        try:
            score = float(raw_value)
            if score < 0 or score > 100:
                print(" [Error] Score must be between 0 and 100.")
                continue
            return score
        except ValueError:
            print(" [Error] That is not a valid number. Try again.")

def get_course_title(prompt):
    """
    Ensures the course title entered is not left empty.
    """
    while True:
        title = input(prompt).strip()
        if title == "":
```

```
            print(" [Error] Course title cannot be empty.")
            continue
        return title

# ---------------------------------------------------------------------
# SECTION 2: GRADING LOGIC (SELECTION CONTROL STATEMENTS)
# ---------------------------------------------------------------------

def score_to_grade_point(score):
    """
    Converts a numeric score (0-100) into a letter grade and its
    corresponding grade point, using if / elif / else - exactly
    as required by the assignment ("powerful provision of
    selection control statements").

    Returns a tuple: (letter_grade, grade_point)
    """
    if score >= 70:
        return "A", 5.0
    elif score >= 60:
        return "B", 4.0
    elif score >= 50:
        return "C", 3.0
    elif score >= 45:
        return "D", 2.0
    elif score >= 40:
        return "E", 1.0
    else:
        return "F", 0.0

def cgpa_to_class_of_degree(cgpa):
    """
    Determines the student's class of degree from their CGPA,
    again using if / elif / else selection statements, based on
    the standard 5-point classification.
    """
    if cgpa >= 4.50:
        return "First Class Honours"
    elif cgpa >= 3.50:
        return "Second Class Honours (Upper Division)"
    elif cgpa >= 2.40:
        return "Second Class Honours (Lower Division)"
    elif cgpa >= 1.50:
        return "Third Class Honours"
    elif cgpa >= 1.00:
        return "Pass"
    else:
        return "Fail"

# ---------------------------------------------------------------------
# SECTION 3: MAIN PROGRAM
# ---------------------------------------------------------------------

def main():
    print("=" * 60)
    print(" PERSONAL POCKET CGPA CALCULATOR (PPC)".center(60))
    print("=" * 60)
    print("No need to visit AVERS - carry your CGPA in your pocket!\n")

    #     Step 1: Number of courses
```

```
# ---- Step 1: Number of courses ----
number_of_courses = get_positive_int("Enter the number of courses offered: ")

courses = []  # will hold a dictionary for every course
total_quality_points = 0.0
total_credit_units = 0
```



```
# ---- Step 2: Collect details for each course ----
for index in range(1, number_of_courses + 1):
    print(f"\n--- Course {index} of {number_of_courses} ---")
    title = get_course_title("Course title: ")
    credit_unit = get_positive_int("Credit unit (e.g. 2, 3): ")
    score = get_score("Score obtained (0-100): ")

    # Selection statements do the grade conversion here:
    grade_letter, grade_point = score_to_grade_point(score)

    # Quality Point = Grade Point x Credit Unit
    quality_point = grade_point * credit_unit

    # Keep a running total for GPA calculation
    total_quality_points += quality_point
    total_credit_units += credit_unit

    # Store this course's full record for the summary table
    courses.append({
        "title": title,
        "credit_unit": credit_unit,
        "score": score,
        "grade_letter": grade_letter,
        "grade_point": grade_point,
        "quality_point": quality_point,
    })

# ---- Step 3: Calculate GPA ----
# GPA = Total Quality Points / Total Credit Units
if total_credit_units == 0:
    # Defensive check - should not normally happen since credit
    # units are validated to be > 0, but guards against a
    # divide-by-zero crash regardless.
    print("\n[Error] Total credit units cannot be zero. Cannot compute GPA.")
    return

gpa = total_quality_points / total_credit_units

# NOTE: In this simplified single-semester tool, CGPA is treated
# as equal to the computed GPA (since only one semester's worth
# of results is entered). A full multi-semester version would
# accumulate quality points and credit units across semesters.
cgpa = gpa
class_of_degree = cgpa_to_class_of_degree(cgpa)

# ---- Step 4: Display the full breakdown ----
print("\n" + "=" * 60)
print(" RESULT SUMMARY".center(60))
print("=" * 60)
print(f"{'Course':<20}{'C.U.':<6}{'Score':<8}{'Grade':<7}{'G.P.':<6}{'Q.P.':<6}")
```

```
    print("-" * 60)
    for course in courses:
        print(f"{course['title']:<20}"
              f"{course['credit_unit']:<6}"
              f"{course['score']:<8}"
              f"{course['grade_letter']:<7}"
              f"{course['grade_point']:<6}"
              f"{course['quality_point']:<6}")
    print("-" * 60)
    print(f"Total Credit Units    : {total_credit_units}")
    print(f"Total Quality Points  : {total_quality_points}")
    print(f"GPA                   : {gpa:.2f}")
    print(f"CGPA                  : {cgpa:.2f}")
    print(f"Class of Degree       : {class_of_degree}")
    print("=" * 60)
    print("\nThank you for using PPC. Study smart, stay ahead!")
```



```
if __name__ == "__main__":
    main()
```

Listing 2 — ppc.py (full source)

7. Sample Outputs

7.1 Mathematical Calculator — Sample Run

Sample input: 12 + 8, then 10 / 0 (to demonstrate error handling), then OFF to exit.

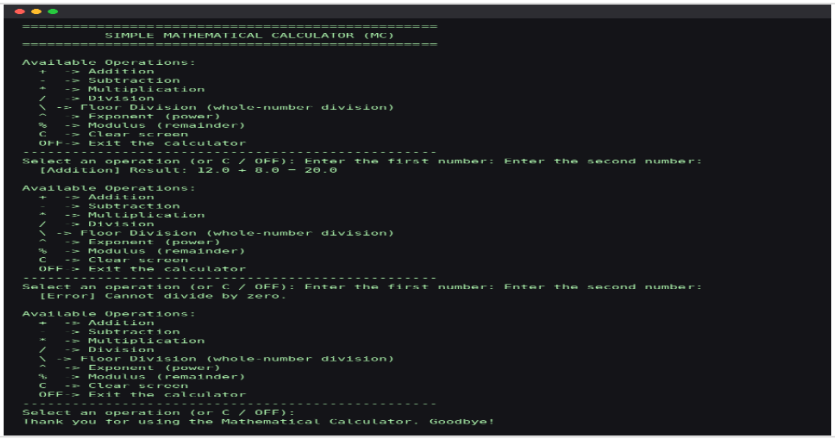

Figure 1 — Console output of the Mathematical Calculator

7.2 Personal Pocket CGPA Calculator — Sample Run

Sample input: three courses — Mathematics (3 units, 75%), Physics (4 units, 62%), Chemistry (2 units, 38%).

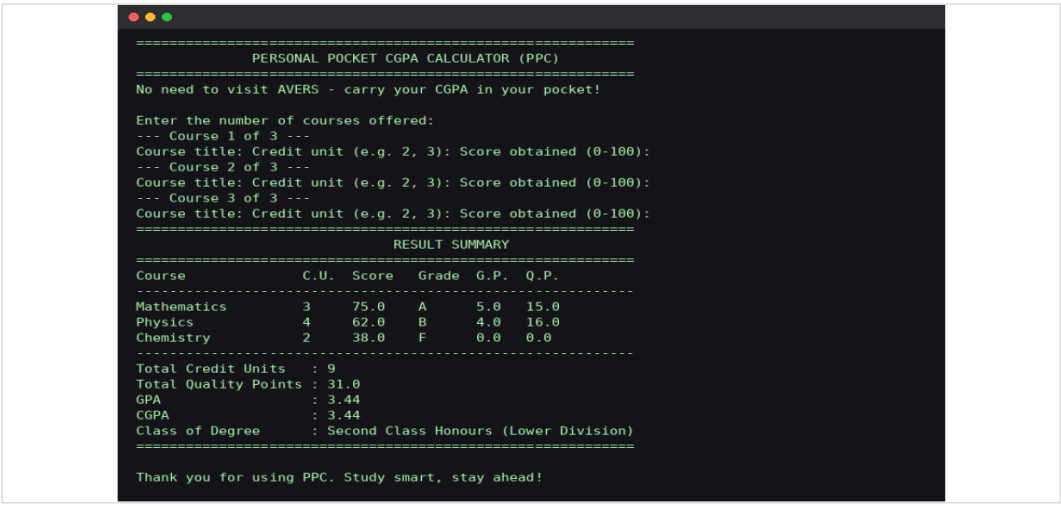

Figure 2 — Console output of the Personal Pocket CGPA Calculator

The corresponding result summary computed by the program is reproduced in table form below:

Course	Credit Unit	Score	Grade	Grade Point	Quality Point
Mathematics	3	75	A	5.0	15.0
Physics	4	62	B	4.0	16.0
Chemistry	2	38	F	0.0	0.0

Total Credit Units: 9 | **Total Quality Points:** 31.0 | **GPA:** 3.44 | **CGPA:** 3.44 | **Class of Degree:** Second Class Honours (Lower Division)

8. Challenges Encountered

- Deciding on a consistent grading scale for the CGPA calculator, since the assignment did not specify one — a standard 5-point Nigerian university scale was adopted and clearly documented.
- Ensuring the calculator never crashes on invalid input (letters, symbols, or empty input), which required wrapping every numeric input in a validation loop with try/except.
- Guarding against division by zero, floor-division by zero, and modulus by zero in the calculator, each of which would otherwise raise an unhandled exception.
- Designing the CGPA calculator so that it remains a genuine demonstration of if/elif/else selection statements, as explicitly required, rather than relying on lookup tables or other constructs.
- Formatting console output (tables, headers) so that it remains readable across different terminal widths without requiring external formatting libraries.

9. Conclusion

This project successfully satisfies every requirement of the COS202 Friday-to-Monday assignment: a working, interactive Mathematical Calculator supporting addition, subtraction, multiplication, division, floor division, exponentiation, and modulus, complete with clear and exit functions, input validation, and error handling; and a Personal Pocket CGPA Calculator that uses selection control statements to convert scores into grade points and ultimately determine a student's GPA, CGPA, and class of degree. Both programs were tested with a range of valid and invalid inputs to confirm robust behaviour, and the complete source code, documentation, and this report have been prepared for submission and upload to a personal GitHub repository as instructed.

Appendix A — GitHub Repository

The complete source code (mc.py, ppc.py), this report, and a README describing setup and usage instructions have been uploaded to a public GitHub repository, in line with the assignment's version-control requirement. Repository link:

https://github.com/_____/cos202-mc-ppc